

گزارشکار آزمایش پنجم سیستم عامل



پارسا احمدی ناو، آریا عازم، مجید صادقی نژاد

پرسش ها

۱. راجع به مفهوم ناحیه مجازی در لینوکس به طور مختصر توضیح داده و آن را با xv6 مقایسه

کنید. صفحه‌های محافظ به چه منظور استفاده می‌شوند؟

در سیستم‌عامل لینوکس، هر فرایند دارای فضای آدرس مجازی است که به بخش‌های مختلفی به نام ناحیه‌های مجازی تقسیم می‌شود. VMA ها ساختارهایی هستند که هرکدام بخشی از فضای آدرس مجازی را توصیف می‌کنند و شامل اطلاعاتی مثل آدرس شروع و پایان، نوع دسترسی، ارتباط با فایل، نوع حافظه است. این اطلاعات در ساختار vm_area_struct ذخیره می‌شوند و کرنل از آنها برای مدیریت حافظه استفاده می‌کند.

در xv6، مفهومی دقیق و مجزایی به نام VMA وجود ندارد. کل فضای آدرس مجازی فرایند در یک آرایه از صفحه‌ها (page table) نگهداری می‌شود. xv6 ساده‌سازی شده است و بخش‌هایی مانند stack، heap، و text توسط برنامه‌نویس و سیستم به‌طور ثابت و ساده مدیریت می‌شود.

لینوکس از یک ساختار انعطاف‌پذیر و پیچیده (VMA) برای مدیریت دقیق‌تر فضای آدرس استفاده می‌کند، ولی xv6 طراحی آموزشی ساده‌تری دارد و کل فضای مجازی در یک جدول صفحه پیوسته و ساده قرار دارد.

صفحه‌های محافظ صفحاتی هستند که در بین ناحیه‌های مختلف حافظه قرار می‌گیرند و معمولاً دارای دسترسی non-accessible هستند. بین heap و stack یک صفحه محافظ قرار می‌گیرد تا در صورت رشد بیش‌ازحد stack، کرنل از طریق page fault متوجه شود. مانع از دسترسی تصادفی یا خرابکارانه بین ناحیه‌های حافظه می‌شود. باعث می‌شود خطاهای دسترسی به حافظه کشف شوند.

۲. چرا ساختار سلسله‌مراتبی منجر به کاهش مصرف حافظه می‌گردد؟

دلیل این موضوع در نحوه‌ی نگهداری اطلاعات آدرس‌دهی است. در ساختار سلسله‌مراتبی جدول صفحه به چند سطح تقسیم می‌شود. جدول‌های پایین‌تر (صفحه‌ها) فقط وقتی ایجاد می‌شوند که نیاز باشد. نیازی نیست کل جدول صفحه را از قبل بسازیم و فقط بخش‌هایی از جدول که واقعا لازم‌اند در حافظه نگه داشته می‌شوند. سیستم‌عامل می‌تواند حافظه را برای جدول‌های فرعی به صورت پویا تخصیص دهد.

۳. محتوای هر بیت یک مدخل ۳۲ بیتی در هر سطح چیست؟ چه تفاوتی میان آن‌ها وجود دارد؟

هر مدخل ۳۲ بیتی در جدول صفحه سلسله‌مراتبی ۲ سطحی شامل آدرس فریم صفحه بعدی یا فریم حافظه فیزیکی بعلاوه بیت‌های کنترلی است. بسته به اینکه در کدام سطح از جدول هستیم، محتوا و معنی دقیق بیت‌ها فرق دارند.

اگر دو سطح داشته باشیم، تفاوت به این صورت است: سطح اول دارای آدرس فریم صفحه بعدی است و سطح دوم دارای آدرس صفحه فیزیکی است. بین دو سطح page directory و page table تفاوت‌هایی هم در بیت‌های کنترلی وجود دارد ولی تفاوت اصلی همین اشاره به page table بعدی و آدرس حافظه فیزیکی است.

۴. تابع kalloc چه حافظه‌ای تخصیص می‌دهد؟ فیزیکی یا مجازی؟

در از یک آرایه از صفحات آزاد استفاده می‌کند که کرنل در زمان راه‌اندازی (boot) آن را آماده کرده است. این تابع یک صفحه ۴KB از حافظه فیزیکی (physical memory) را اختصاص می‌دهد. مقدار بازگشتی kalloc یک آدرس فیزیکی است که نشان‌دهنده‌ی مکان واقعی صفحه در RAM است.

۵. تابع mappages چه کاربردی دارد؟

این تابع وظیفه دارد یک یا چند آدرس مجازی را به آدرس‌های فیزیکی متناظرشان map کند. به عبارت دیگر، mappages ورودی‌های جدول صفحات را طوری مقداردهی می‌کند که آدرس‌های مجازی مشخص‌شده به آدرس‌های فیزیکی داده‌شده اشاره کنند. برای هر صفحه از بازه‌ی مشخص‌شده، بررسی می‌کند آیا نگاشتی از قبل وجود دارد یا نه. اگر وجود ندارد، ورودی صفحه را ایجاد می‌کند و آدرس فیزیکی و بیت‌های دسترسی را در آن قرار می‌دهد.

۶. آیا تمام قسمت‌های حافظه مجازی از آدرس صفر تا `proc->sz` از طریق کاربر قابل استفاده اند؟
خیر، تمام بخش‌های حافظه مجازی از آدرس صفر تا `proc->sz` لزوماً برای کاربر قابل استفاده نیستند. در `xv6`، فضای حافظه مجازی هر فرایند از آدرس صفر شروع شده و تا `proc->sz` ادامه دارد. این مقدار نشان‌دهنده‌ی انتهای فعلی فضای مجازی تخصیص داده‌شده به فرایند است. فقط بخش‌هایی از این بازه قابل استفاده توسط کاربر هستند که واقعاً **map شده اند** و دارای **دسترسی کاربر** هستند. ممکن است آدرسی به حافظه فیزیکی نگاشت نشده باشد. در بعضی صفحات هم بیت `PTE_U` که نشان‌دهنده دسترسی کاربر هست، غیرفعال است.

۷. راجع به تابع `walk pgdir` توضیح دهید. این تابع چه عمل سخت افزاری را شبیه سازی می‌کند؟
این تابع در مکانیزم ترجمه آدرس مجازی به فیزیکی نقش دارد. برای یک آدرس مجازی خاص، ورودی جدول صفحه‌ی متناظر (`PTE`) را پیدا می‌کند یا در صورت نیاز و درخواست، آن را ایجاد می‌کند. اگر `PTE` مربوطه موجود باشد، اشاره‌گر به آن `PTE` را برمی‌گرداند. در غیر صورت آن را ایجاد می‌کند. `walkpgdir` شبیه‌سازی سخت‌افزاری عملکرد **MMU** در معماری `x86` را انجام می‌دهد. واحد `MMU` سپس خودش به‌طور سخت‌افزاری این ترجمه را انجام می‌دهد.

۸. توابع `allocvm` و `mappages` که در ارتباط با حافظه مجازی هستند را توضیح دهید.
`int allocvm(pde_t *pgdir, uint oldsz, uint newsz)`
تابع `allocvm` فضای حافظه مجازی جدیدی از `oldsz` تا `newsz` برای فرایند ایجاد می‌کند، برای هر صفحه یک صفحه فیزیکی با `kalloc` تخصیص می‌دهد و آن را با استفاده از `mappages` نگاشت می‌کند.

`int mappages(pde_t *pgdir, void *va, uint size, uint pa, int perm)`
صفحات حافظه مجازی را به صفحات فیزیکی با مجوزهای مشخص در جدول صفحه نگاشت می‌کند.

۹. شیوه بارگذاری برنامه در حافظه توسط فراخوانی سیستمی `exec` را شرح دهید.

در فراخوانی سیستمی `exec`، ابتدا فایل اجرایی با `namei` پیدا و باز می‌شود. سپس هدر `ELF` خوانده می‌شود تا بخش‌های قابل بارگذاری مشخص شوند. فضای آدرس مجازی جدید با `setupvm` ساخته می‌شود. هر بخش از فایل اجرایی با استفاده از `allocvm` و `loadvm` در حافظه مجازی بارگذاری می‌شود. دو صفحه برای پشته تخصیص داده می‌شود که یکی از آن‌ها صفحه محافظ است. آرگومان‌های خط فرمان در بالای پشته کپی و تنظیم می‌شوند. سپس مقادیر

مناسب به ثبات‌های trapframe مثل eip و esp داده می‌شود تا اجرای برنامه جدید آغاز شود. در پایان، فضای آدرس قبلی آزاد شده و فضای جدید جایگزین آن می‌شود.

۱۰. پیاده‌سازی مانیتورها معمولاً در برگیرنده مفهومی به اسم متغیر شرطی برای signaling است، اما در این پروژه از شما خواسته نشده چنین ساز و کاری پیاده کنید. این مکانیزم را توضیح دهید و شرح دهید پیاده‌سازی آن در معماری xv6 چه مشکلاتی ممکن است به همراه داشته باشد؟

مکانیزم متغیر شرطی (Condition Variable) یکی از اجزای اصلی در پیاده‌سازی مانیتورها است. این متغیر به فرایندها یا نخ‌ها اجازه می‌دهد تا در صورت برقرار نبودن یک شرط، به صورت اتمی قفل را آزاد کرده و منتظر بمانند تا آن شرط برقرار شود. سپس توسط نخ دیگری که آن شرط را برقرار کرده، با عملیات signal یا broadcast بیدار شوند و دوباره قفل را به دست آورند.

در ادامه مشکلات این مکانیزم در xv6 را توضیح می‌دهیم:

۱. سادگی هسته:

xv6 یک سیستم‌عامل آموزشی ساده است و از ابتدا پشتیبانی داخلی برای متغیرهای شرطی، صف‌های انتظار پیچیده یا سیستم‌های هماهنگ‌سازی پیشرفته ندارد. این باعث می‌شود ساختن wait و signal نیاز به تغییرات اساسی در هسته داشته باشد.

۲. نبود سیستم صف‌بندی مستقل برای شرایط:

در xv6 نخ‌ها معمولاً روی صف‌های ساده منتظر می‌مانند. پیاده‌سازی متغیر شرطی نیاز به صف‌های مجزا برای هر شرط دارد که با وضعیت قفل‌ها تعامل دقیق داشته باشند.

۳. پیچیدگی آزادسازی اتمی قفل:

در عملیات wait باید قفل به‌طور اتمی آزاد شود و نخ به صف انتظار برود. این کار در xv6 پیچیده است چون عملیات release و sleep به صورت جداگانه اجرا می‌شوند و اتمی بودن آن‌ها تضمین نشده مگر با تغییرات اساسی در کد.

۴. امکان بروز بن‌بست (Deadlock):

اگر مراقبت کافی در پیاده‌سازی انجام نشود (مثلاً اگر نخ بیدار شده نتواند قفل را پس بگیرد)، امکان رخ دادن بن‌بست وجود دارد.